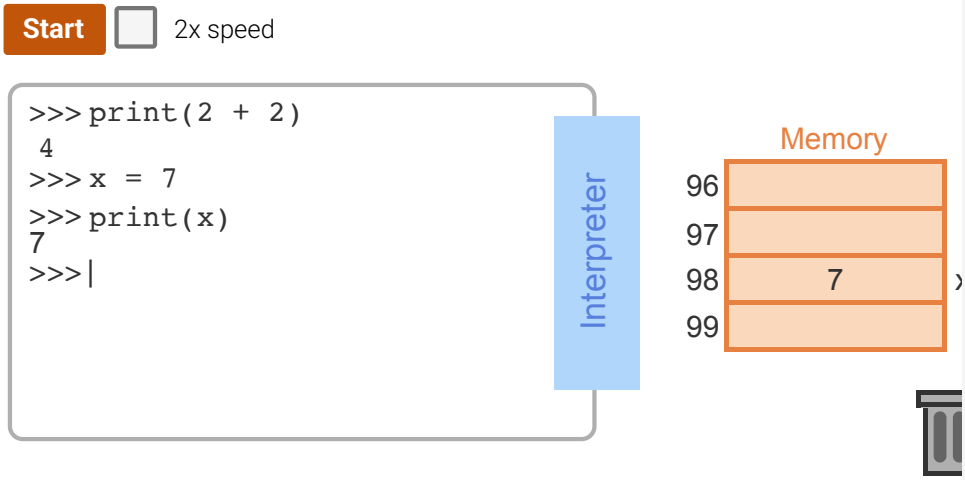
2.3 Objects

object

An object represents a value and is automatically created by the ir
line of code. For example, executing `x = 4` creates a new object t

PARTICIPATION ACTIVITY 2.3.1: Creating new objects.



Captions

1. The interpreter creates a new object with the value 4. The object is stored somewhere in m
2. Once 4 is printed, the object is no longer needed and is thrown away.
3. New object created: "x" references the object stored in address 98.
4. Objects are retrieved from memory when needed.

garbage collection

Deleting unused objects is an automatic process called garbage c
memory space.

Name binding

Name binding is the process of associating names with interprete
have more than one name bound to it, and every name is always b
object. Name binding occurs whenever an assignment statement
demonstrated below.

PARTICIPATION ACTIVITY 2.3.2: Manipulating variables.

Start 2x speed

file.py

```
bob_salary = 25000
tom_salary = 30000
bob_salary = tom_salary
tom_salary = tom_salary * 1.2
total_salaries = bob_salary + tom_salary
```

Interpreter

Names

bob_salary

tom_salary

total_salaries

Captions

1. bob_salary object is created by the interpreter.
2. tom_salary object is created by the interpreter.
3. bob_salary is assigned tom_salary, and the 25000 object is garbage collected.
4. tom_salary is assigned tom_salary * 1.2.
5. total_salaries object is created by the interpreter and is assigned bob_salary + tom_salary.

Value

Value: A value such as "20", "abcdef", or "55".

Type

Type: The type of the object, such as integer or string.

Identity

Identity: A unique identifier that describes the object.

Figure 2.3.1: Printing displays an object's value.

```
x = 2 + 2     # Create a new object with a value of 4, referenc
print(x)      # Print the value of the object.
print(5)
```

type()

The built-in function type() returns the type of an object.

Mutability

Mutability indicates whether the object's value is allowed to be cha

immutable

Integers and strings are immutable; meaning integer and string va

Figure 2.3.2: Using type() to print an object's type.

```
x = 2 + 2           # Create a new object with a value of 4, referenced
print(type(x))      # Print the type of the object.

print(type("ABC"))  # Create and print the type of a string object.
```

id()

Python provides a built-in function id() that gives the value of an ol

Figure 2.3.3: Using id() to print an object's identity.

```
x = 2 + 2           # Create a new object with a value of 4, referen
print(id(x))        # Print the identity (memory address) of the x o

print(id("ABC"))    # Create and print the identity of a string ("AB
```

Try 2.3.1: Experimenting with objects.

Run the following code and observe the results of type() and id(). Create a new object called "age id and type of the new object.

Run

main.py

```
birthday_year = 1986
birthday_month = "April"
birthday_day = 22

print("birthday_year -->")
print(" value:", birthday_year)
print(" type:", type(birthday_year))
print(" id:", id(birthday_year))

print("\nbirthday_month -->")
print(" value:", birthday_month)
print(" type:", type(birthday_month))
print(" id:", id(birthday_month))

print("\nbirthday_day -->")
print(" value:", birthday_day)
print(" type:", type(birthday_day))
print(" id:", id(birthday_day))

# Your code goes here
```

CONSOLE

Model Solution